
EduNavigator Technologies

EduNavigator- CCNY Intelligent Academic Navigation System Design Report

Version <2.0>

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

Revision History

Date	Version	Description	Author
03/23/2026	<1.0>	EduNavigator – CCNY Intelligent Academic Navigation System	Mim D., Zara R., Mashiyat J., David H., Guillermo M.
04/23/2026	<2.0>	EduNavigator – CCNY Intelligent Academic Navigation System	Mim D., Zara R., Mashiyat J., David H., Guillermo M.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

Table of Contents

1. Introduction
2. All Use-Cases
 - 2.1 Use-Case Scenarios and Diagrams and Petri Nets
3. E-R Diagrams
 - 3.1 Diagram for Entire System
4. Detailed Design
 - 4.1 Pseudocode for All
5. Major GUI Screens
 - 5.1 One Functionality
6. Memos of group meetings
7. Address of GitHub Repo

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

Design Report

1. Introduction

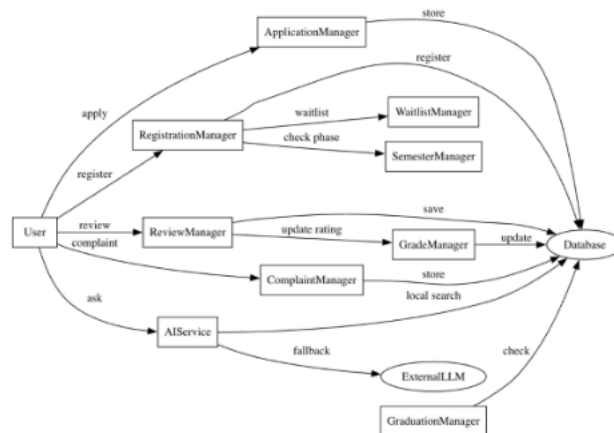
The EduNavigator – CCNY Intelligent Academic Navigation System is an AI-enabled academic management platform designed to support students, instructors, registrars, and visitors in managing academic and administrative processes. The system integrates functionalities such as application processing, course registration, grading, reviews, complaints, and graduation tracking into a centralized interface.

This design report presents the internal structure and interaction of system components required to implement the functionalities defined in the Software Requirements Specification. The system is organized into multiple manager classes, each responsible for a specific subsystem, including ApplicationManager, RegistrationManager, ReviewManager, GradeManager, and AIService.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
--	----------------

Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

1.1 Collaboration Class Diagram



- 1.
2. Explanation of the Diagram
3. The collaboration class diagram provides an overall picture of how the main components of the EduNavigator system interact. Users interact with different manager classes such as ApplicationManager, RegistrationManager, ReviewManager, ComplaintManager, and AIService to perform various operations.
4. Each manager class communicates with the central database to store and retrieve system data. The RegistrationManager coordinates with the WaitlistManager and SemesterManager to manage course enrollment and enforce registration rules. The ReviewManager interacts with the GradeManager to update course ratings and enforce warning policies. The GraduationManager checks eligibility based on academic records.
5. The AIService retrieves answers from the local database and, if necessary, sends queries to an external large language model (LLM) as a fallback. Overall, the diagram illustrates the high-level collaboration between system components and shows how different parts of the system work together to support academic operations.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

2. All Use-Cases

2.1 Use-Case Scenarios and Diagrams and Petri Nets

1. Use Case 1: Submit Student Application

Primary actor: Guest

Goal: Apply to the college as a student

1. Normal scenario

1. Guest opens the public application page.
2. Guest enters full name.
3. Guest selects **Student** as the role.
4. Guest enters GPA.
5. Guest clicks **Submit Application**.
6. System stores the application with status **Pending**.
7. System shows a success message.

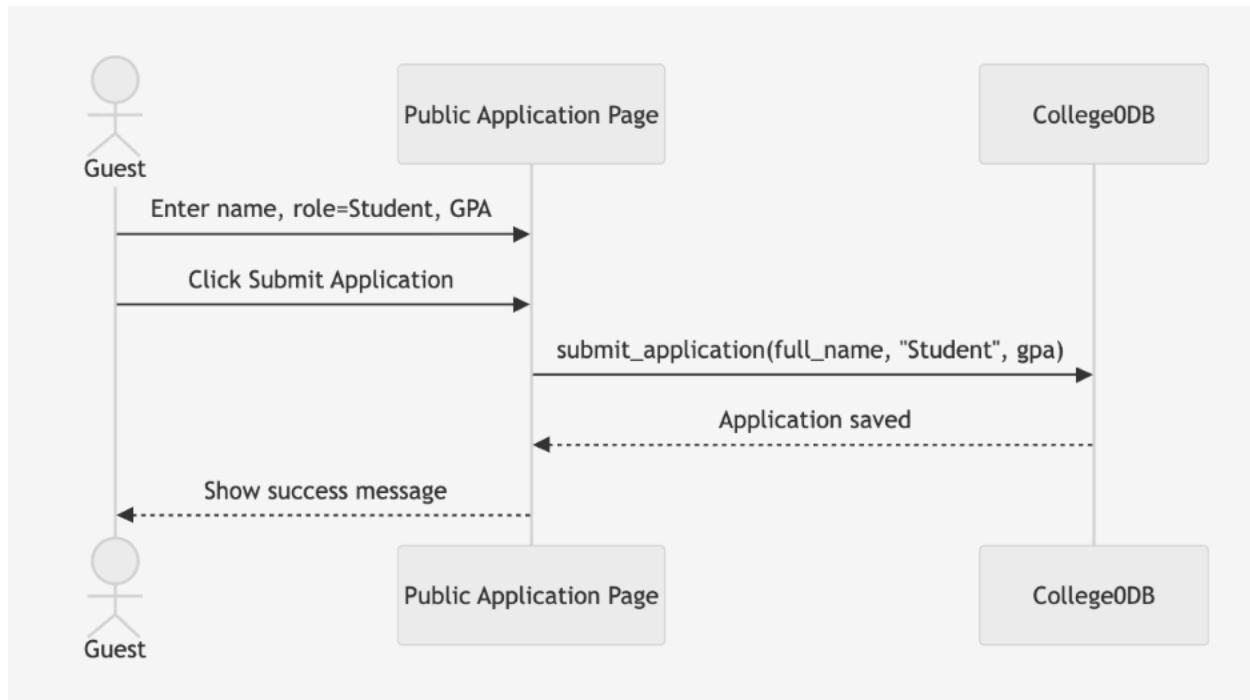
2. Exceptional scenario

1. If the full name field is empty, the system shows an error.
2. If GPA is not numeric, the system shows an invalid GPA error.
3. If the database save fails, the application is not stored.

Postcondition: A pending student application is stored in the system.

3. Sequence diagram

This use case is implemented through the public application form and `submit_application(...)`.



EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

2. Use Case 2: Submit Instructor Application

Primary actor: Guest

Goal: Apply to the college as an instructor

1. Normal scenario

1. Guest opens the public application page.
2. Guest enters full name.
3. Guest selects **Instructor** as the role.
4. Guest leaves GPA as 0.00 or enters a value.
5. Guest clicks **Submit Application**.
6. System stores the application with status **Pending**.
7. System shows a success message.

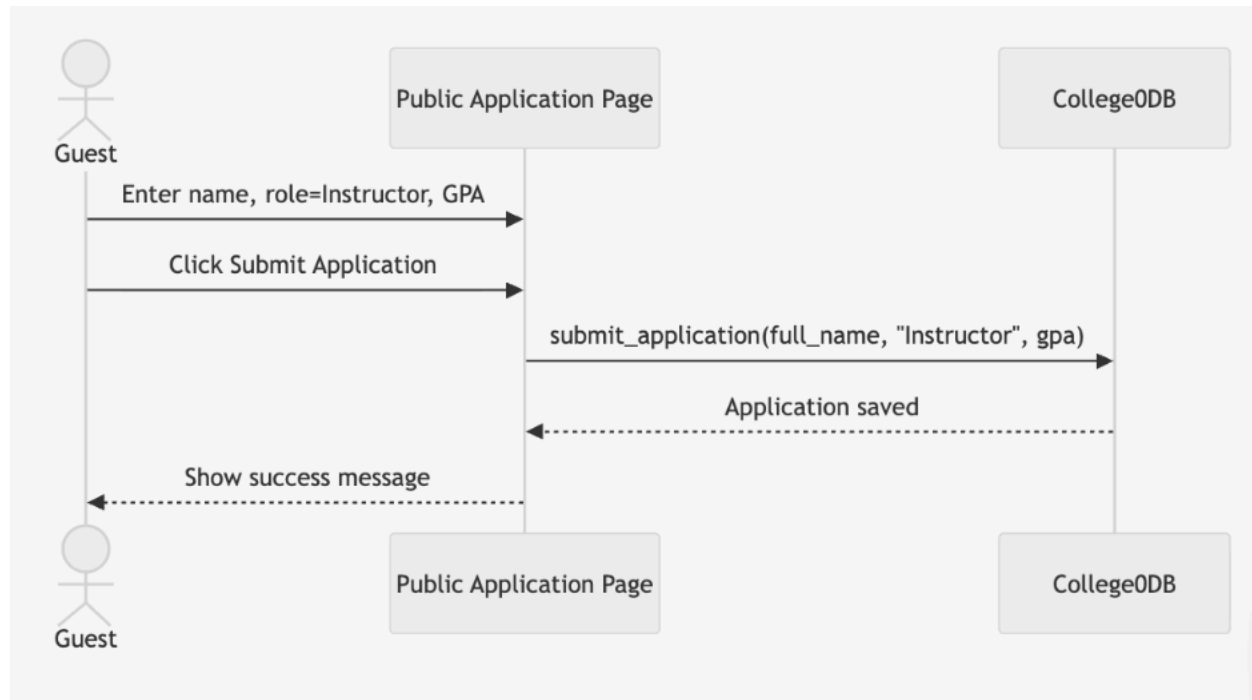
2. Exceptional scenario

1. If the full name field is empty, the system shows an error.
2. If GPA is not numeric, the system shows an invalid GPA error.
3. If the database save fails, the application is not stored.

Postcondition: A pending instructor application is stored in the system.

3. Sequence diagram

This follows the same application submission flow as the student application, but with role set to Instructor.



EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

3. Use Case 3: Login

Primary actor: Student / Instructor / Registrar

Goal: Access the system dashboard

1. Normal scenario

1. User opens the login page.
2. User enters username and password.
3. User clicks **Login**.

4. System authenticates the credentials.
5. System starts the session.
6. System opens the dashboard for that user role.
7. If the account requires a password change, the system opens the password-change window.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

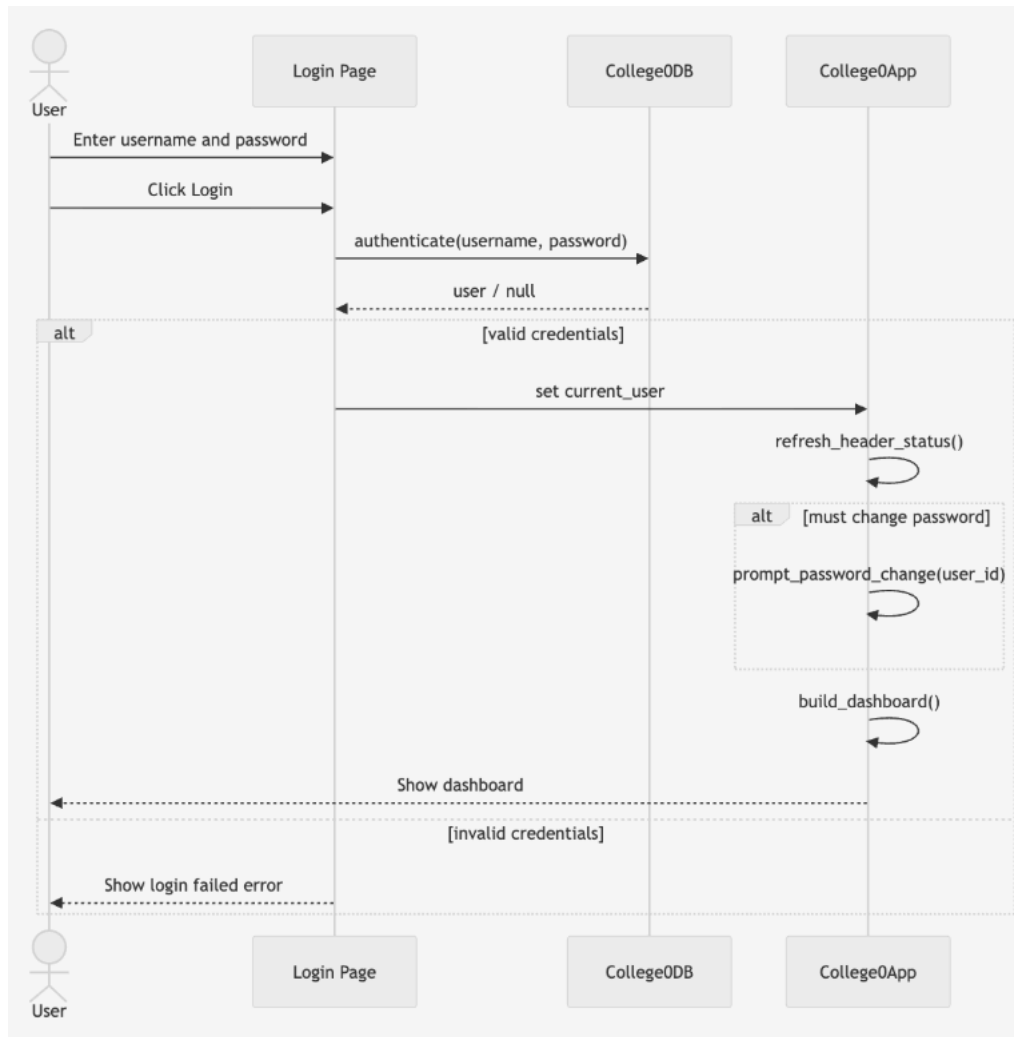
2. Exceptional scenario

1. If the username or password is incorrect, the system shows **Login failed**.
2. If authentication returns no user, access is denied.
3. If the session cannot be created, the dashboard does not open.

Postcondition: The user is logged in and sees the appropriate dashboard.

3. Sequence diagram

Login is handled by `authenticate(...)`, followed by dashboard loading and optional password change.



4. Use Case 4: Change Password

Primary actor: Student

Goal: Replace temporary password after first login

1. Normal scenario

1. Student logs in using a temporary password.
2. System detects that password change is required.
3. System opens the password-change window.
4. Student enters a new password.
5. Student clicks **Save Password**.
6. System updates the password.
7. System removes the must-change-password flag.
8. System shows a success message.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

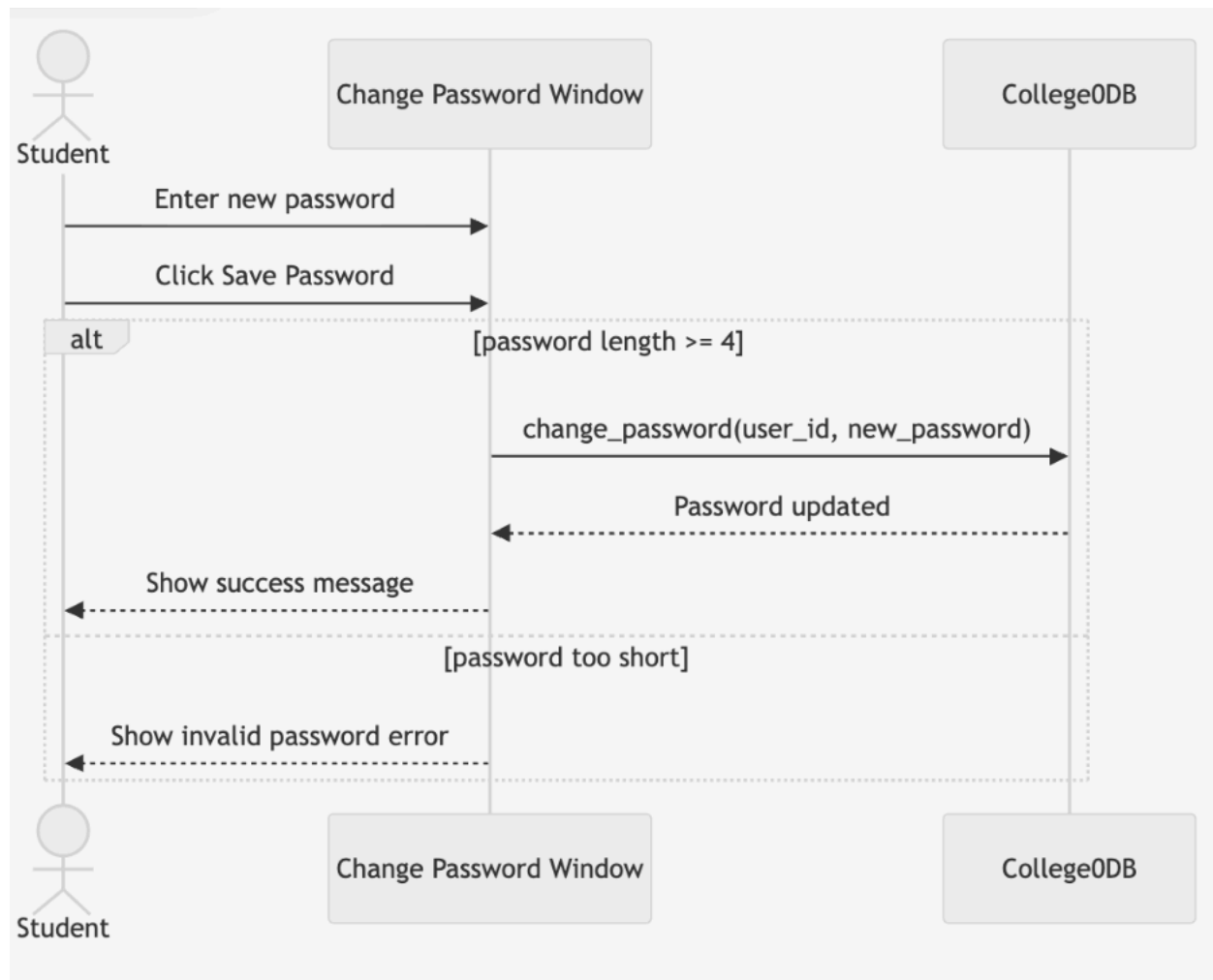
2. Exceptional scenario

1. If the new password is shorter than 4 characters, the system shows an error.
2. If the password update fails, the old password remains unchanged.

Postcondition: The account now uses the new password.

3. Sequence diagram

Password change is handled by `change_password(...)`.



EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

5. Use Case 9: Submit Review / Instructor Rating

Primary actor: Student

Goal: Submit a rating and review for a class

1. Normal scenario

1. Student logs in.
2. Student enters class ID.
3. Student selects a star rating.

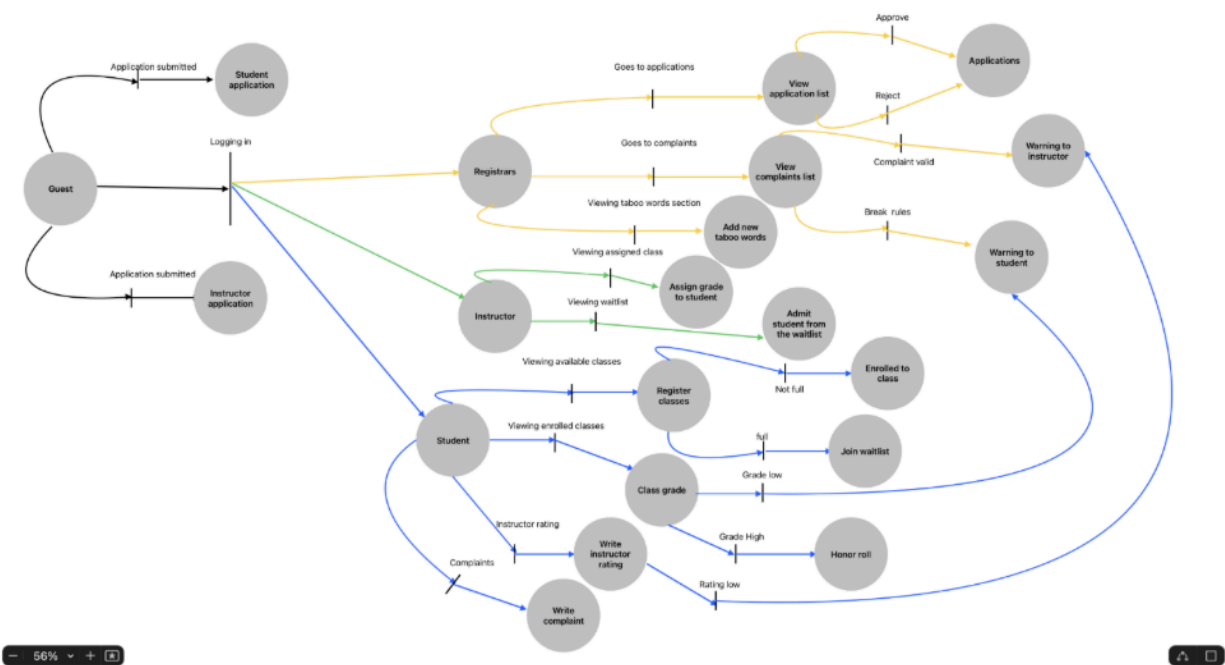
4. Student types a review.
5. Student clicks **Submit Review**.
6. System checks the review against taboo words.
7. System stores the review.
8. System updates instructor ratings.
9. System shows a confirmation message.

2. Exceptional scenario

1. If class ID is invalid, the system shows an error.
2. If stars are invalid, the system shows an error.
3. If the student already reviewed that class, the system rejects the submission.
4. If the review contains 1 or 2 taboo words, the review is stored but the student gets a warning.
5. If the review contains 3 or more taboo words, the review is hidden and the student gets extra warnings.
6. If database insertion fails, the review is not saved.

Postcondition: The review is stored, hidden, or rejected; warnings may be added.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	



6. Use Case 10: File Complaint

Primary actor: Student

Goal: Report another student or instructor

1. Normal scenario

1. Student logs in.
2. Student opens the complaint section.
3. Student selects a target user.
4. Student enters complaint details.
5. Student clicks **Submit Complaint**.
6. System stores the complaint with status **Open**.
7. System shows a confirmation message.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

2. Exceptional scenario

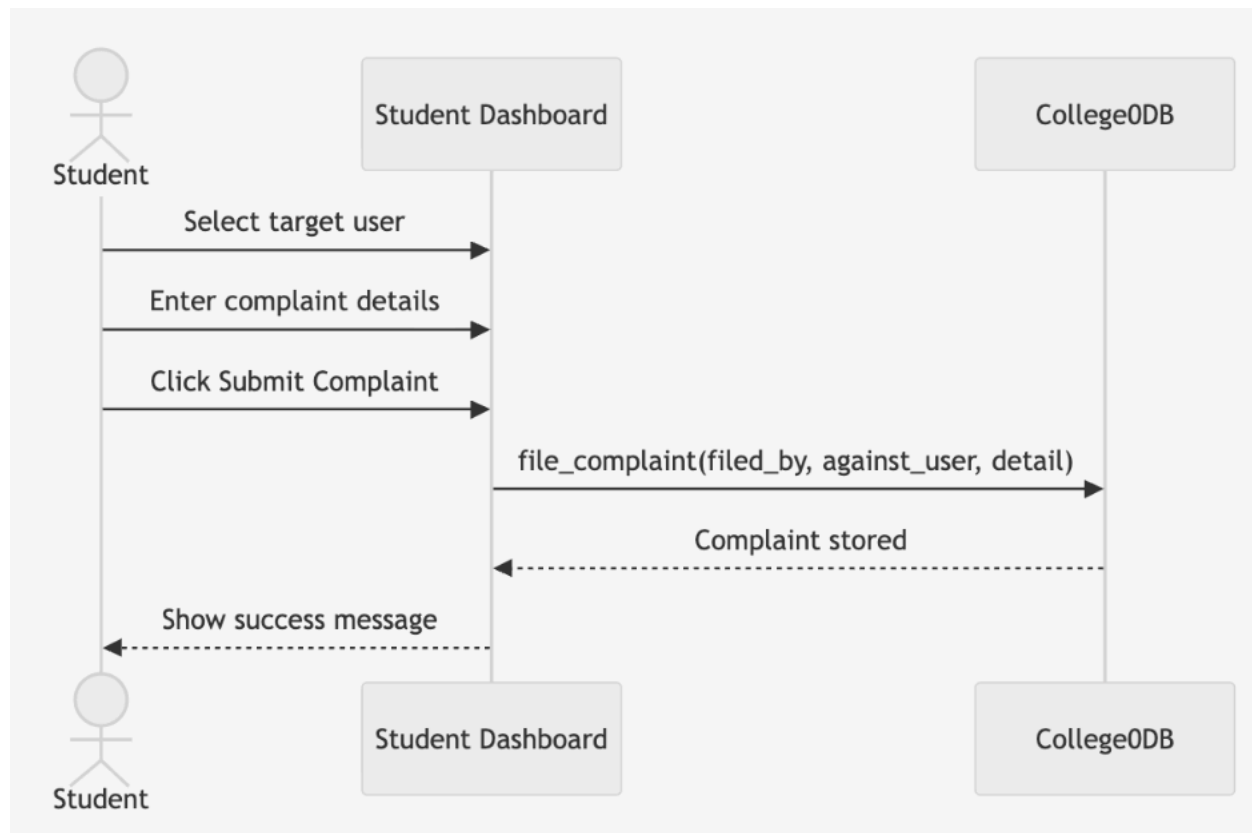
1. If no target is selected, the system shows an error.

2. If complaint detail is empty, the system shows an error.
3. If database insertion fails, the complaint is not saved.

Postcondition: A new complaint is stored for registrar review.

3. Sequence diagram

Complaints are saved through `file_complaint(...)`.



EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

7. Use Case 11: Add Taboo Word

Primary actor: Registrar

Goal: Add a forbidden word for review moderation

1. Normal scenario

1. Registrar logs in.
2. Registrar opens the taboo words section.
3. Registrar enters a new word.
4. Registrar clicks **Add**.
5. System stores the word in lowercase form.
6. System refreshes the dashboard.

2. Exceptional scenario

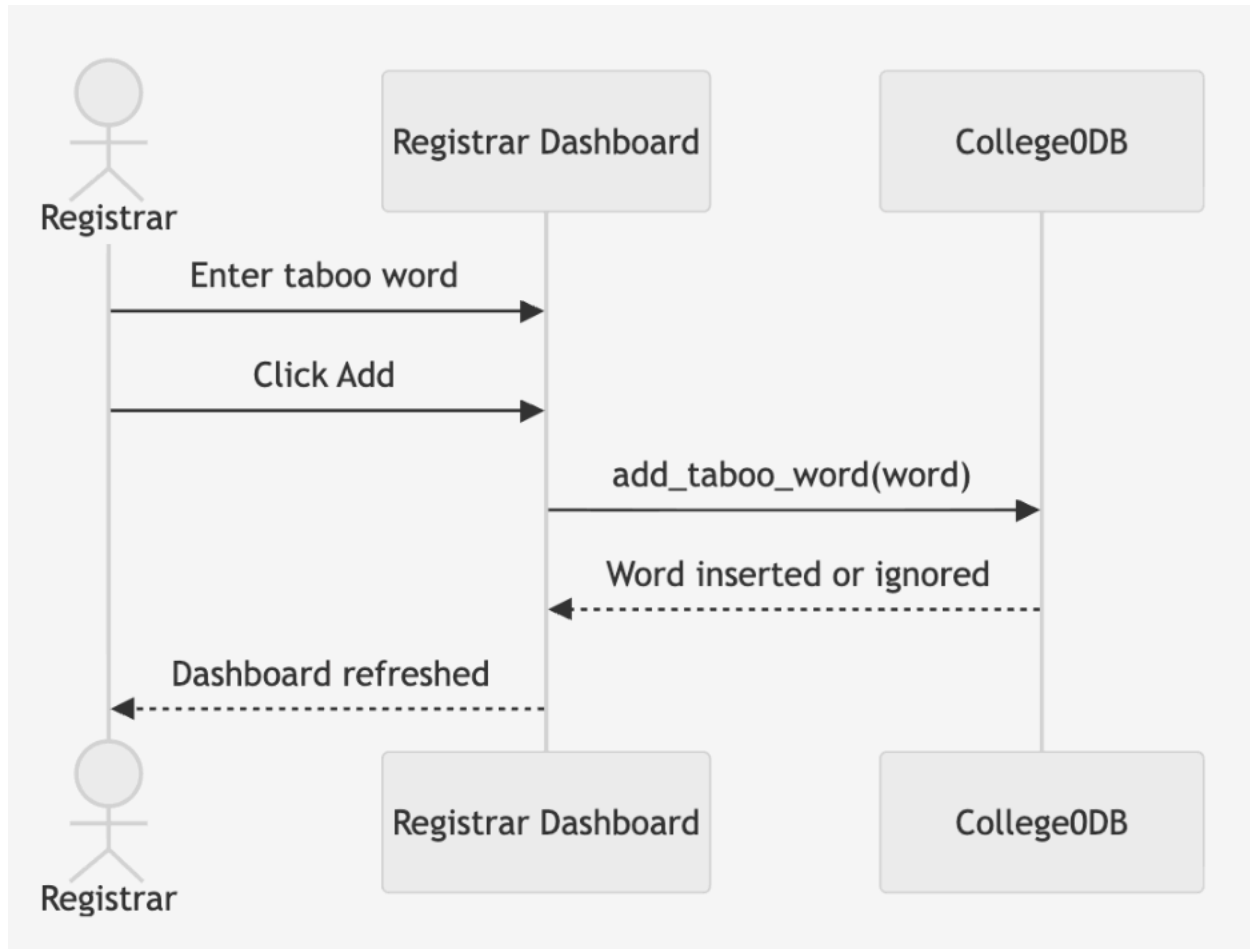
1. If the word is blank, nothing is added.
2. If the word already exists, the system ignores the duplicate.
3. If the database update fails, the word is not saved.

Postcondition: The taboo word list is updated.

3. Sequence diagram

This use case calls `add_taboo_word(...)`.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	



8. Use Case 12: Resolve Complaint / Issue Warning

Primary actor: Registrar

Goal: Resolve a complaint and assign a warning

1. Normal scenario

1. Registrar logs in.
2. Registrar opens the complaints list.
3. Registrar selects a complaint.
4. Registrar chooses either **Warn Accused** or **Warn Filer**.
5. System updates the selected user's warning count.

6. System updates the complaint status to resolved.
7. System refreshes the dashboard.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

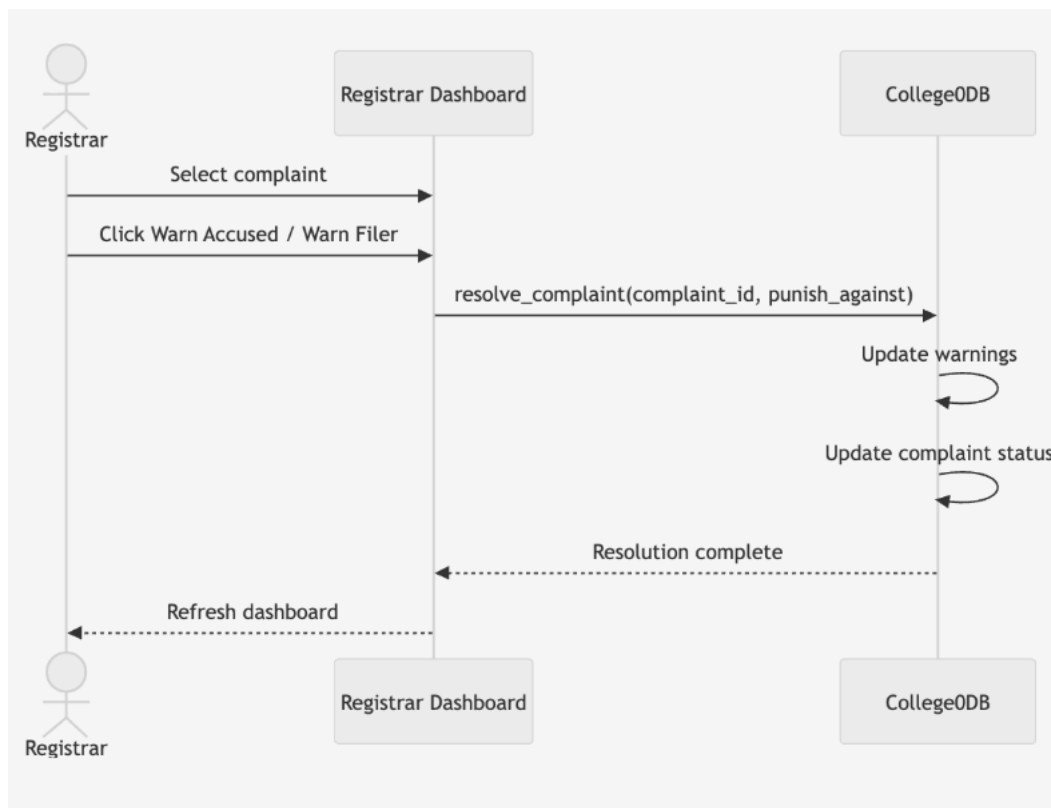
2. Exceptional scenario

1. If no complaint is selected, no action is performed.
2. If the complaint is not found, resolution fails.
3. If the database update fails, the complaint remains unresolved.

Postcondition: The complaint is resolved and a warning is issued to one party.

3. Sequence diagram

Complaint resolution is handled by `resolve_complaint(...)`.



EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

9. Use Case 13: Assign Grade

Primary actor: Instructor

Goal: Assign a grade to a student in a class

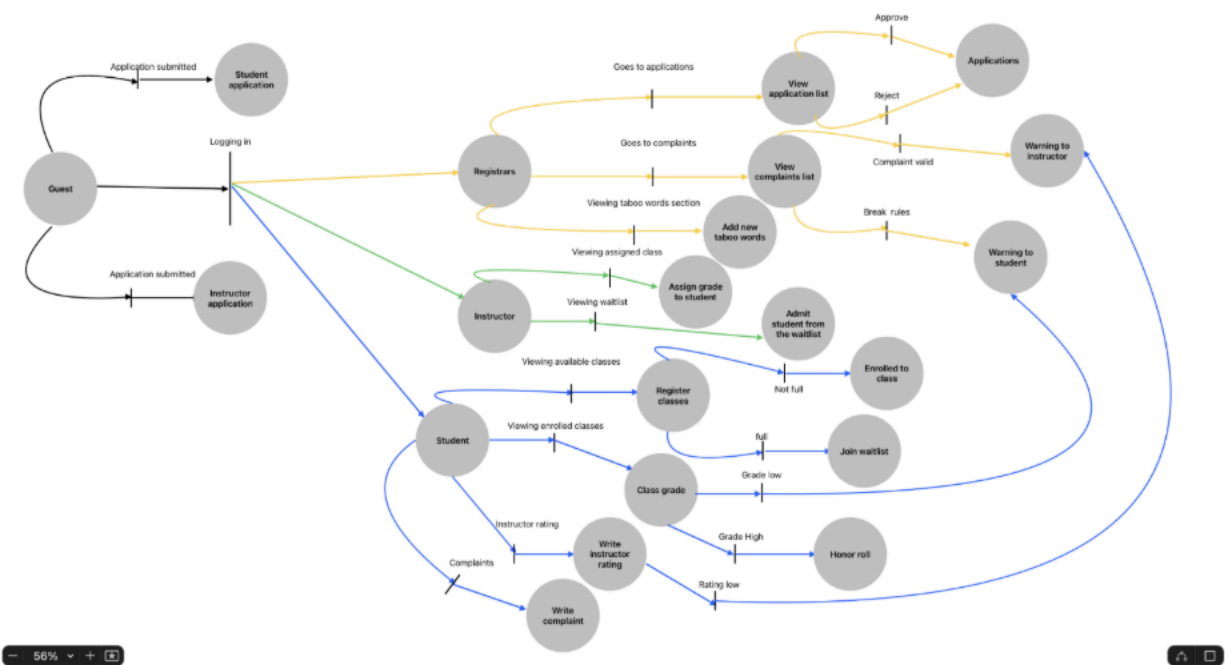
1. Normal scenario

1. Instructor logs in.
2. Instructor opens assigned classes.
3. Instructor selects a class.
4. System displays enrolled students.
5. Instructor selects a student.
6. Instructor selects a grade.
7. Instructor clicks **Assign Grade**.
8. System updates the registration record.
9. System recalculates GPA.
10. System updates honor roll or suspension/warning status when needed.
11. System shows a success message.

2. Exceptional scenario

1. If no class is selected, no grading occurs.
2. If no student is selected, no grading occurs.
3. If the registration record is invalid, the grade cannot be saved.
4. If GPA recalculation fails, academic status may not update correctly.

Postcondition: The grade is stored and student academic status may change.



10. Use Case 14: Admit Waitlisted Student

Primary actor: Instructor

Goal: Move a waitlisted student into an enrolled seat

1. Normal scenario

1. Instructor logs in.
2. Instructor selects one of their classes.
3. System displays the waitlist.
4. Instructor selects a waitlisted student.
5. Instructor clicks **Admit Selected Student**.
6. System checks class capacity.
7. If a seat is available, the system moves the student from waitlist to registrations.
8. System removes the student from the waitlist.
9. System shows a confirmation message.

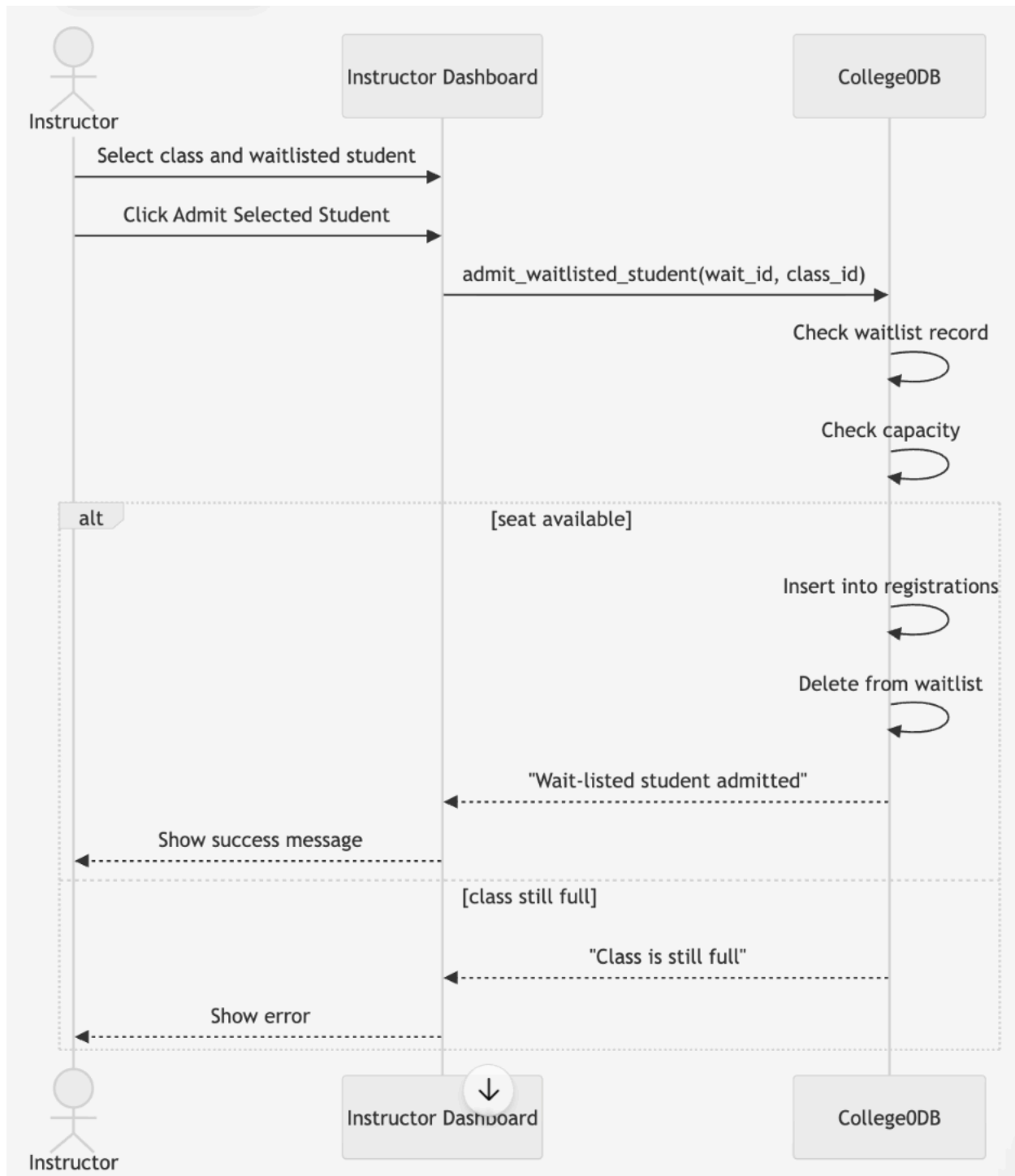
2. Exceptional scenario

1. If no class is selected, no action is performed.
2. If no waitlisted student is selected, no action is performed.
3. If the waitlist record is not found, admission fails.
4. If the class is still full, the student cannot be admitted.
5. If registration insertion fails, the student remains on the waitlist.

Postcondition: The student is either admitted or remains on the waitlist.

3. Sequence diagram

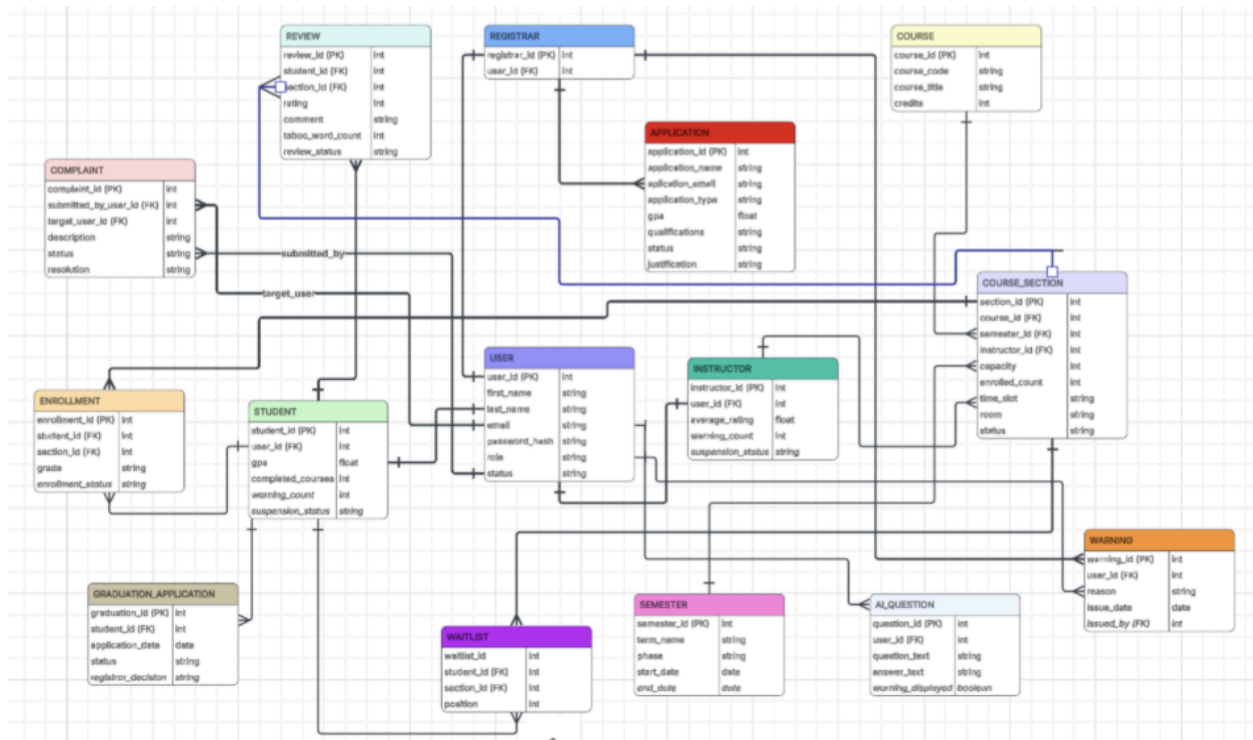
Admission is handled by `admit_waitlisted_student(...)`.



EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

3. E-R Diagrams

3.1 Diagram for entire system



EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Software Requirements Specification	Date: 04/23/2026
EduNavigator-SRS-v2.0	

4. Detailed Design

4.1 Pseudocode for All

This section presents the detailed design of the EduNavigator system. Each class is responsible for a major subsystem of the application. For every method, the input, output, and main functionality are described using pseudocode. These methods are derived from the system requirements and from the current implementation of the EduNavigator prototype.

4. 4.1 ApplicationManager

Method: submitApplication(fullName, roleApplied, gpa)

Input:

fullName
roleApplied
Gpa

Output:

application submission status

Pseudocode:

METHOD submitApplication(fullName, roleApplied, gpa)

IF fullName is empty THEN

RETURN "Applicant name required"

IF roleApplied is not "Student" AND roleApplied is not "Instructor" THEN

RETURN "Invalid application role"

```
IF roleApplied = "Student" AND gpa < 0 THEN  
RETURN "Invalid GPA"
```

```
INSERT application record into Applications table  
SET status = "Pending"
```

```
RETURN "Application submitted successfully"  
END METHOD
```

Method: getApplications()

Input:

none

Output:

list of application records

Pseudocode:

```
METHOD getApplications()  
    applications <- SELECT all records FROM Applications  
    ORDER applications by status and application id descending
```

```
RETURN applications  
END METHOD
```

Method: approveApplication(appId, justification)

Input:

appId

justification

Output:

approval result and generated credentials

Pseudocode:

```
METHOD approveApplication(appId, justification)  
    application <- findApplication(appId)
```

```
IF application does not exist THEN
```



```

    RETURN "Application not found"

IF application.status != "Pending" THEN
    RETURN "Application already processed"

    UPDATE application status to "Approved"
    SAVE justification

    IF application.roleApplied = "Student" THEN
        username <- generateStudentUsername(appId)
        password <- generateTemporaryStudentPassword()
        CREATE user account with role "Student"
        CREATE student profile using application GPA
    ELSE
        username <- generateInstructorUsername(appId)
        password <- generateTemporaryInstructorPassword()
        CREATE user account with role "Instructor"
        CREATE instructor profile
    END IF

    RETURN "Approved with generated username and password"
END METHOD

```

Method: rejectApplication(appId, justification)

Input:

appId
justification

Output:

rejection status

Pseudocode:

```

METHOD rejectApplication(appId, justification)
    application <- findApplication(appId)

    IF application does not exist THEN
        RETURN "Application not found"

```

```
UPDATE application status to "Rejected"  
SAVE justification
```

```
RETURN "Application rejected"  
END METHOD
```

5. 4.2 RegistrationManager

Method: getAvailableClasses()

Input:

none

Output:

list of available class sections

Pseudocode:

```
METHOD getAvailableClasses()  
    classes <- SELECT all non-cancelled class sections  
    INCLUDE course code, title, meeting time, instructor, capacity, and enrolled count  
    SORT by course code  
  
    RETURN classes  
END METHOD
```

Method: getStudentRegistrations(studentId)

Input:

studentId

Output:

list of classes currently registered by the student

Pseudocode:

```
METHOD getStudentRegistrations(studentId)  
    registrations <- SELECT all class registrations  
    WHERE student_id = studentId  
  
    RETURN registrations  
END METHOD
```

Method: registerCourse(studentId, sectionId)

Input:

studentId

sectionId

Output:

enrollment status

Pseudocode:

METHOD registerCourse(studentId, sectionId)

 section <- findSection(sectionId)

 student <- findStudent(studentId)

 IF section does not exist THEN

 RETURN "Class not found"

 IF student does not exist THEN

 RETURN "Student not found"

 IF section.periodState != "Registration" THEN

 RETURN "Registration closed"

 IF student.suspended = true THEN

 RETURN "Student suspended"

 selectedCount <- countCurrentRegistrations(studentId)

 IF selectedCount >= 4 THEN

 RETURN "Maximum course load exceeded"

 IF hasTimeConflict(studentId, section.meetingTime) THEN

 RETURN "Time conflict detected"

 IF isAlreadyRegistered(studentId, sectionId) THEN

 RETURN "Already registered"

 IF isRetakeNotAllowed(studentId, section.courseId) THEN

 RETURN "Retake not allowed"

 enrolledCount <- countSectionEnrollment(sectionId)

```
        IF enrolledCount < section.capacity THEN
        createEnrollment(studentId, sectionId)
        RETURN "Enrolled successfully"
        ELSE
        addToWaitlist(studentId, sectionId)
        RETURN "Added to waitlist"
    END METHOD
```

Method: hasTimeConflict(studentId, meetingTime)

Input:

studentId
meetingTime

Output:

true or false

Pseudocode:

```
METHOD hasTimeConflict(studentId, meetingTime)
    currentRegistrations <- getStudentRegistrations(studentId)

    FOR each registration in currentRegistrations DO
    IF registration.meetingTime = meetingTime THEN
    RETURN true
    END IF
    END FOR

    RETURN false
END METHOD
```

Method: isRetakeNotAllowed(studentId, courseId)

Input:

studentId
courseId

Output:

true or false

Pseudocode:

```
METHOD isRetakeNotAllowed(studentId, courseId)
    previousGrades <- getPreviousGrades(studentId, courseId)

    IF previousGrades is empty THEN
        RETURN false

    IF student previously passed the course THEN
        RETURN true

    RETURN false
END METHOD
```

Method: createEnrollment(studentId, sectionId)

Input:

studentId
sectionId

Output:

enrollment creation status

Pseudocode:

```
METHOD createEnrollment(studentId, sectionId)
    INSERT new record into Registrations table
    SET initial grade = blank

    RETURN "Enrollment created"
END METHOD
```

6. 4.3 WaitlistManager

Method: addToWaitlist(studentId, sectionId)

Input:

studentId
sectionId

Output:

waitlist status

Pseudocode:

```
METHOD addToWaitlist(studentId, sectionId)
    IF student is already on waitlist for this section THEN
        RETURN "Already on waitlist"

    INSERT record into Waitlist table
    SAVE current timestamp

    RETURN "Added to waitlist"
END METHOD
```

Method: getWaitlist(sectionId)

Input:

sectionId

Output:

ordered list of waitlisted students

Pseudocode:

```
METHOD getWaitlist(sectionId)
    waitlist <- SELECT waitlist records for sectionId
    ORDER BY created_at ascending

    RETURN waitlist
END METHOD
```

Method: admitWaitlistedStudent(waitId, sectionId)

Input:

waitId
sectionId

Output:

admission status

Pseudocode:

```
METHOD admitWaitlistedStudent(waitId, sectionId)
    waitRecord <- findWaitlistRecord(waitId, sectionId)
```

```

    IF waitRecord does not exist THEN
    RETURN "Waitlist record not found"

    section <- findSection(sectionId)
    enrolledCount <- countSectionEnrollment(sectionId)

    IF enrolledCount >= section.capacity THEN
    RETURN "Class is still full"

    createEnrollment(waitRecord.studentId, sectionId)
    DELETE waitRecord from Waitlist

    RETURN "Waitlisted student admitted"
END METHOD

```

7. 4.4 ReviewManager

Method: submitReview(studentId, sectionId, rating, comment)

Input:

studentId
sectionId
rating
comment

Output:

review status

Pseudocode:

```

METHOD submitReview(studentId, sectionId, rating, comment)
    IF gradesAlreadyPosted(sectionId) THEN
    RETURN "Review not allowed"

    IF hasAlreadyReviewed(studentId, sectionId) THEN
    RETURN "Review already submitted"

    tabooCount <- countTabooWords(comment)

    IF tabooCount >= 3 THEN
    hiddenComment <- "[Hidden due to taboo language]"
    issueWarnings(studentId, 2)

```

```

saveReview(studentId, sectionId, rating, comment, hiddenComment, true)
updateCourseRating(sectionId)
updateInstructorRating(sectionId)
    RETURN "Review rejected"
    ELSE IF tabooCount >= 1 THEN
        comment <- censorTabooWords(comment)
        issueWarnings(studentId, 1)
    END IF

```

```

saveReview(studentId, sectionId, rating, comment, comment, false)
updateCourseRating(sectionId)
updateInstructorRating(sectionId)

```

```

    RETURN "Review submitted"
END METHOD

```

Method: gradesAlreadyPosted(sectionId)

Input:
sectionId

Output:
true or false

Pseudocode:
METHOD gradesAlreadyPosted(sectionId)
 gradeRecords <- getGradesForSection(sectionId)

```

        FOR each record in gradeRecords DO
            IF record.grade is not empty THEN
                RETURN true
            END IF
        END FOR

```

```

    RETURN false
END METHOD

```

Method: countTabooWords(comment)

Input:

comment

Output:

number of taboo words found

Pseudocode:

```
METHOD countTabooWords(comment)
  tabooWords <- getTabooWords()
  words <- split comment into individual words
  count <- 0

  FOR each word in words DO
    cleanedWord <- normalize(word)
    IF cleanedWord is in tabooWords THEN
      count <- count + 1
    END IF
  END FOR

  RETURN count
END METHOD
```

Method: censorTabooWords(comment)

Input:

comment

Output:

censored comment

Pseudocode:

```
METHOD censorTabooWords(comment)
  tabooWords <- getTabooWords()
  words <- split comment into individual words
  visibleWords <- empty list

  FOR each word in words DO
    cleanedWord <- normalize(word)
    IF cleanedWord is in tabooWords THEN
      ADD masked version of word to visibleWords
    ELSE
```

```
    ADD word to visibleWords
  END IF
END FOR
```

```
  censoredComment <- join visibleWords into text
  RETURN censoredComment
END METHOD
```

Method: saveReview(studentId, sectionId, rating, originalText, visibleText, hiddenFlag)

Input:

studentId
sectionId
rating
originalText
visibleText
hiddenFlag

Output:

save status

Pseudocode:

```
METHOD saveReview(studentId, sectionId, rating, originalText, visibleText, hiddenFlag)
  INSERT review record into Reviews table
  STORE rating, original text, visible text, and hidden flag

  RETURN "Review saved"
END METHOD
```

Method: updateCourseRating(sectionId)

Input:

sectionId

Output:

updated course rating

Pseudocode:

```
METHOD updateCourseRating(sectionId)
  reviews <- getVisibleReviewsForSection(sectionId)
  averageRating <- computeAverage(reviews.stars)
```

STORE averageRating for reporting purposes

RETURN averageRating
END METHOD

Method: getReviews()

Input:
none

Output:
list of reviews

Pseudocode:
METHOD getReviews()
 reviews <- SELECT all reviews with course, student, stars, and visible text
 ORDER BY course code

 RETURN reviews
END METHOD

8. 4.5 GradeManager

Method: getStudentsInClass(sectionId)

Input:
sectionId

Output:
list of enrolled students and grades

Pseudocode:
METHOD getStudentsInClass(sectionId)
 students <- SELECT all registrations for sectionId
 INCLUDE student name and current grade

 RETURN students
END METHOD

Method: assignGrade(registrationId, grade)

Input:

registrationId
grade

Output:

grade assignment status

Pseudocode:

```
METHOD assignGrade(registrationId, grade)
    IF grade is not in allowedGradeSet THEN
        RETURN "Invalid grade"

    UPDATE registration record
    SET grade = grade

    recalculateGPA()
    RETURN "Grade assigned successfully"
END METHOD
```

Method: recalculateGPA()

Input:

none

Output:

updated GPA values for students

Pseudocode:

```
METHOD recalculateGPA()
    gradePointsMap <- {A:4.0, A-:3.7, B+:3.3, B:3.0, B-:2.7, C+:2.3, C:2.0, D:1.0, F:0.0}

    students <- getAllStudents()

    FOR each student in students DO
        grades <- getCompletedGrades(student.id)
        points <- convert grades to grade points using gradePointsMap

        IF points is empty THEN
            gpa <- 0.0
```

```

ELSE
  gpa <- round(sum(points) / count(points), 2)
END IF

honorRoll <- false
IF gpa > 3.75 THEN
  honorRoll <- true
END IF

UPDATE student profile with gpa, overall_gpa, and honorRoll

  IF gpa < 2.0 AND student has completed grades THEN
    suspendStudent(student.id)
  ELSE IF gpa >= 2.0 AND gpa <= 2.25 THEN
    issueWarnings(student.id, 1)
  END IF
END FOR

RETURN "GPA recalculated"
END METHOD

```

Method: updateInstructorRating(sectionId)

Input:
sectionId

Output:
updated instructor average rating

Pseudocode:
 METHOD updateInstructorRating(sectionId)
 instructorId <- getInstructorForSection(sectionId)
 allReviews <- getReviewsForInstructor(instructorId)
 averageRating <- computeAverage(allReviews.stars)

UPDATE instructor profile average rating

```

  IF averageRating < 2.0 THEN
    issueWarnings(instructorId, 1)
  END IF

```

```
        RETURN averageRating
    END METHOD
```

Method: suspendStudent(studentId)

Input:
studentId

Output:
suspension status

Pseudocode:

```
METHOD suspendStudent(studentId)
    UPDATE user record
    SET suspended = true

    RETURN "Student suspended"
END METHOD
```

Method: issueWarnings(userId, count)

Input:
userId
count

Output:
warning update status

Pseudocode:

```
METHOD issueWarnings(userId, count)
    currentWarnings <- getCurrentWarnings(userId)
    newWarningCount <- currentWarnings + count

    UPDATE user record
    SET warnings = newWarningCount

    IF newWarningCount >= 3 THEN
        markUserForSuspensionReview(userId)
    END IF
```

```
    RETURN "Warnings updated"  
END METHOD
```

9. 4.6 ComplaintManager

Method: fileComplaint(filedBy, againstUser, detail)

Input:

filedBy
againstUser
detail

Output:

complaint filing status

Pseudocode:

```
METHOD fileComplaint(filedBy, againstUser, detail)  
    IF detail is empty THEN  
        RETURN "Complaint detail required"  
  
    INSERT complaint record into Complaints table  
    SET status = "Open"  
  
    RETURN "Complaint submitted"  
END METHOD
```

Method: getComplaints()

Input:

none

Output:

list of complaint records

Pseudocode:

```
METHOD getComplaints()  
    complaints <- SELECT all complaint records  
    INCLUDE filer name, accused name, detail, and status  
    ORDER BY complaint id descending  
  
    RETURN complaints
```

END METHOD

Method: resolveComplaint(complaintId, punishAgainst)

Input:

complaintId
punishAgainst

Output:

complaint resolution status

Pseudocode:

```
METHOD resolveComplaint(complaintId, punishAgainst)
    complaint <- findComplaint(complaintId)
```

```
    IF complaint does not exist THEN
        RETURN "Complaint not found"
```

```
    IF punishAgainst = true THEN
        issueWarnings(complaint.againstUser, 1)
        statusText <- "Resolved - warning issued to accused"
    ELSE
        issueWarnings(complaint.fileedBy, 1)
        statusText <- "Resolved - warning issued to filer"
    END IF
```

```
    UPDATE complaint status to statusText
```

```
    RETURN "Complaint resolved"
END METHOD
```

10. 4.7 SemesterManager

Method: checkRegistrationPeriod(sectionId)

Input:

sectionId

Output:

true or false

Pseudocode:

```
METHOD checkRegistrationPeriod(sectionId)
    section <- findSection(sectionId)

    IF section.periodState = "Registration" THEN
        RETURN true
    ELSE
        RETURN false
END METHOD
```

Method: updateSectionPeriod(sectionId, newPeriod)

Input:

sectionId
newPeriod

Output:

update status

Pseudocode:

```
METHOD updateSectionPeriod(sectionId, newPeriod)
    validPeriods <- ["Class Setup", "Registration", "Running", "Grading"]

    IF newPeriod is not in validPeriods THEN
        RETURN "Invalid period"

    UPDATE section record
    SET periodState = newPeriod

    RETURN "Section period updated"
END METHOD
```

Method: cancelUnderEnrolledSection(sectionId)

Input:

sectionId

Output:

cancellation status

Pseudocode:

```
METHOD cancelUnderEnrolledSection(sectionId)
    enrollmentCount <- countSectionEnrollment(sectionId)
    minimumRequired <- getMinimumEnrollmentThreshold()

    IF enrollmentCount < minimumRequired THEN
        UPDATE section record
        SET cancelled = true

    notifyRegisteredStudents(sectionId)
    RETURN "Section cancelled"
    END IF

    RETURN "Section remains active"
END METHOD
```

11. 4.8 GraduationManager

Method: applyForGraduation(studentId)

Input:

studentId

Output:

graduation application status

Pseudocode:

```
METHOD applyForGraduation(studentId)
    eligibility <- checkGraduationEligibility(studentId)

    IF eligibility = true THEN
        createGraduationApplication(studentId)
        RETURN "Graduation application submitted"
    ELSE
        issueWarnings(studentId, 1)
        RETURN "Graduation requirements not met"
    END IF
END METHOD
```

Method: checkGraduationEligibility(studentId)

Input:

studentId

Output:

true or false

Pseudocode:

```
METHOD checkGraduationEligibility(studentId)
    completedCourses <- countPassedCourses(studentId)
    gpa <- getStudentOverallGPA(studentId)

    IF completedCourses < 8 THEN
        RETURN false

    IF gpa < minimumGraduationGPA THEN
        RETURN false

    RETURN true
END METHOD
```

Method: createGraduationApplication(studentId)

Input:

studentId

Output:

application creation status

Pseudocode:

```
METHOD createGraduationApplication(studentId)
    INSERT graduation application record
    SET status = "Pending Review"

    RETURN "Graduation application created"
END METHOD
```

Method: approveGraduation(studentId)

Input:

studentId

Output:

approval status

Pseudocode:

```
METHOD approveGraduation(studentId)
    IF checkGraduationEligibility(studentId) = false THEN
        RETURN "Student not eligible"

    UPDATE student record
    SET graduation status = "Graduated"

    RETURN "Graduation approved"
END METHOD
```

12. 4.9 AIService

Method: answerQuestion(userId, questionText)

Input:

userId

questionText

Output:

answer text

Pseudocode:

```
METHOD answerQuestion(userId, questionText)
    localAnswer <- searchLocalKnowledgeBase(questionText)

    IF localAnswer is not null THEN
        saveAIQuery(userId, questionText, localAnswer, "LocalDB", false)
        RETURN localAnswer
    END IF

    llmAnswer <- queryExternalLLM(questionText)
    warningMessage <- " This answer may contain inaccuracies."
    saveAIQuery(userId, questionText, llmAnswer, "LLM", true)

    RETURN llmAnswer + warningMessage
END METHOD
```

Method: searchLocalKnowledgeBase(questionText)

Input:

questionText

Output:

local answer or null

Pseudocode:

```
METHOD searchLocalKnowledgeBase(questionText)
  queryEmbedding <- convertQuestionToEmbedding(questionText)
  matchingRecords <- searchVectorDatabase(queryEmbedding)

  IF matchingRecords is empty THEN
    RETURN null

  bestAnswer <- extractBestMatchingAnswer(matchingRecords)
  RETURN bestAnswer
END METHOD
```

Method: queryExternalLLM(questionText)

Input:

questionText

Output:

generated answer

Pseudocode:

```
METHOD queryExternalLLM(questionText)
  SEND questionText to external language model service
  RECEIVE generated response

  RETURN generated response
END METHOD
```

Method: saveAIQuery(userId, questionText, answerText, sourceType, warningFlag)

Input:

userId

questionText
answerText
sourceType
warningFlag

Output:

save status

Pseudocode:

```
METHOD saveAIQuery(userId, questionText, answerText, sourceType, warningFlag)
    INSERT query log into AI query history table
    STORE user, question, answer, source type, and warning flag

    RETURN "AI query saved"
END METHOD
```

13. 4.10 UserManager

Method: authenticate(username, password)

Input:

username
password

Output:

user record or authentication failure

Pseudocode:

```
METHOD authenticate(username, password)
    user <- findUserByUsernameAndPassword(username, password)

    IF user does not exist THEN
        RETURN "Invalid username or password"

    RETURN user
END METHOD
```

Method: changePassword(userId, newPassword)

Input:

userId
newPassword

Output:

password change status

Pseudocode:

```
METHOD changePassword(userId, newPassword)
    IF length(newPassword) < minimumPasswordLength THEN
        RETURN "Password too short"

    UPDATE user record
    SET password = newPassword
    SET mustChangePassword = false

    RETURN "Password changed successfully"
END METHOD
```

Method: getUserSummary(userId)

Input:

userId

Output:

user summary information

Pseudocode:

```
METHOD getUserSummary(userId)
    summary <- SELECT user basic data, warnings, suspension status,
    GPA data, and instructor rating data
    WHERE user id = userId

    RETURN summary
END METHOD
```

Method: getUsersByRole(role)

Input:

role

Output:

list of users with the specified role

Pseudocode:

METHOD getUsersByRole(role)

 users <- SELECT all users WHERE role = role
 ORDER BY full name

 RETURN users

END METHOD

14. 4.11 ReportingManager

Method: publicRankings()

Input:

none

Output:

highest GPA students, highest-rated classes, and lowest-rated classes

Pseudocode:

METHOD publicRankings()

 topStudents <- SELECT students ordered by overall GPA descending

 topClasses <- SELECT visible course reviews ordered by average stars descending

 lowClasses <- SELECT visible course reviews ordered by average stars ascending

 RETURN topStudents, topClasses, lowClasses

END METHOD

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026

5. Major GUI Screens

5.1 One Functionality

CCNY

EduNavigator - CCNY Intelligent Academic Navigation System
CCNY-inspired visual theme

Session: Guest mode

MENU

- Dashboard
- Apply as Student
- Apply as Instructor
- Login
- Help
- Exit

Demo Access

- Registrar: registrar / admin123
- Student: s1001 / temp123
- Instructor: i2001 / teach123

One-window local GUI
Same project logic, isolated branding

Welcome to EduNavigator CCNY

An AI-enabled college management system with a CCNY-inspired campus theme.

TOP	LOW	GPA	CCNY
Top Rated Classes * Advanced Python (4.8) * Data Science (4.6) * Creative Writing (4.5)	Lowest Rated Classes * Intro to Algebra (2.3) * History 101 (2.5) * Art Appreciation (2.6)	Highest GPA Students * Alice Rivera - GPA 3.8 * Camila Torres - GPA 3.4 * Brian Chen - GPA 2.9 * Daniel Park - GPA 2.3	College Highlights * Small college community * Hands-on learning * AI-enabled assistance

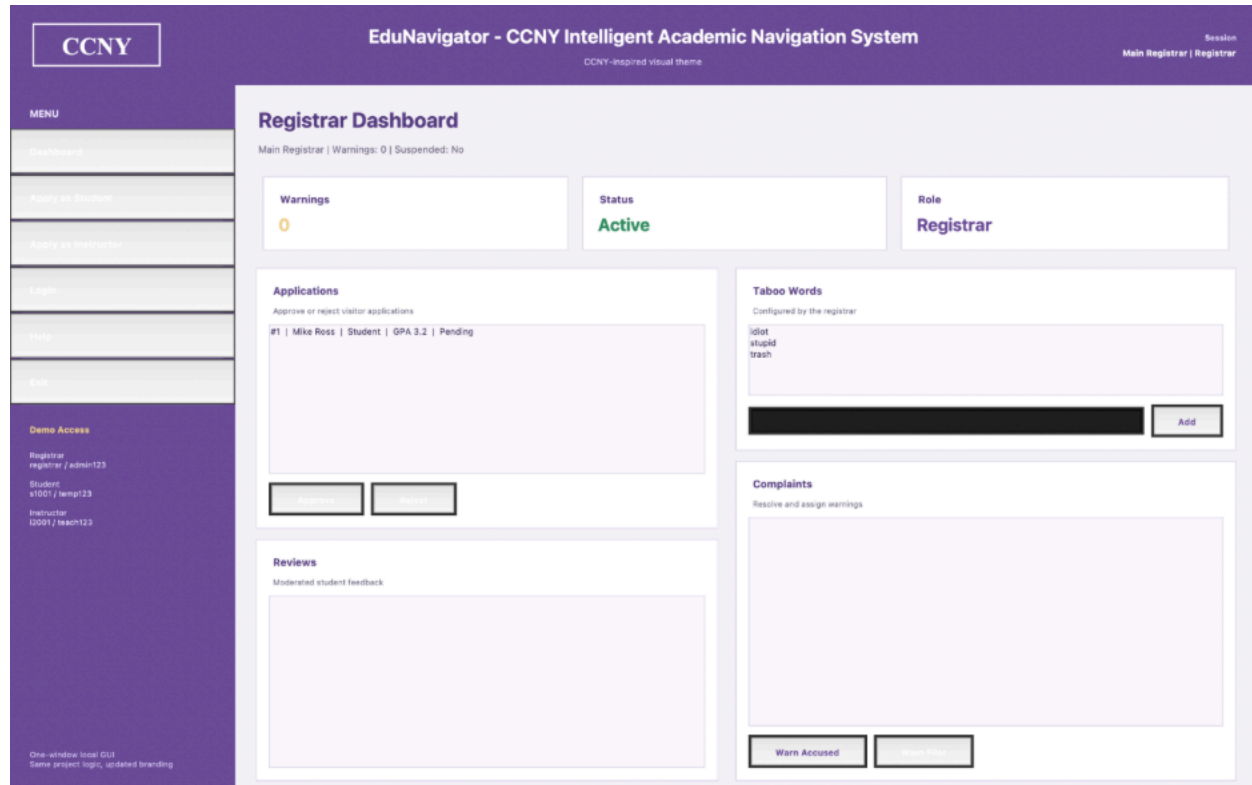
Prospective students and instructors can apply to College0 using the form below.

Visitor Application
Apply as a student or instructor while keeping the same project workflow.

Full Name: Apply As: GPA:

Student applications use GPA. Instructor applications can leave GPA at 0.00.

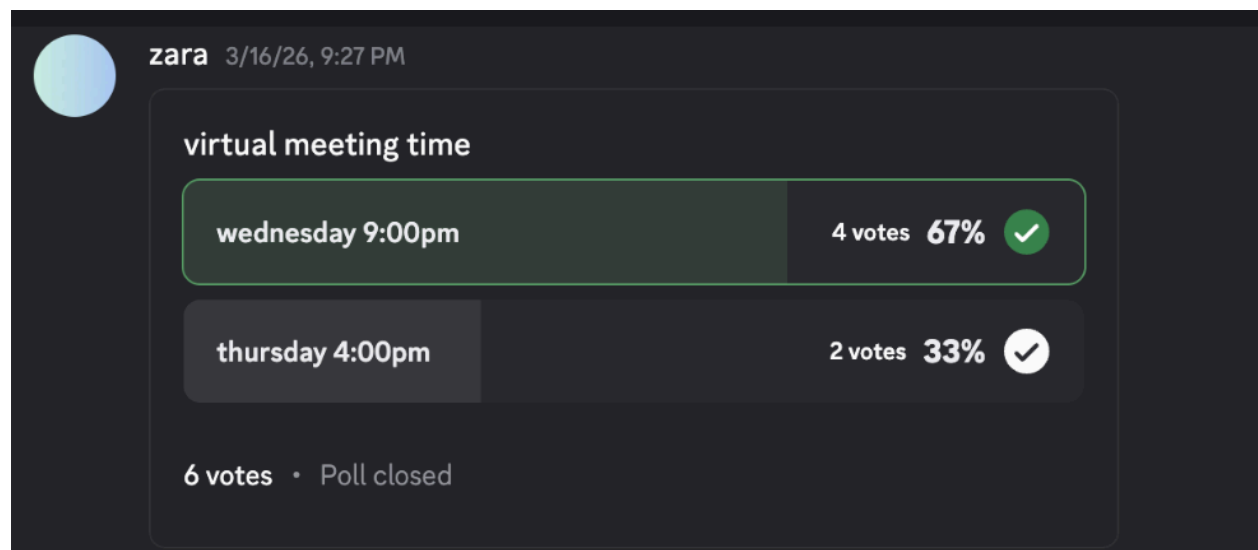
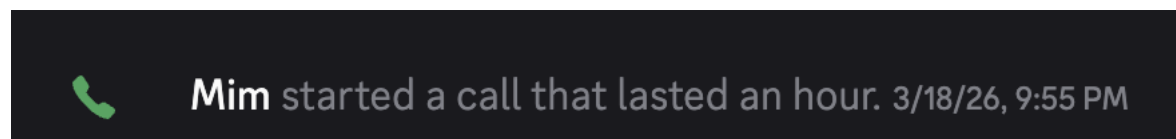
In the image above it shows the main menu of the system called EduNavigator, the role that is presented in the image is guest which is the visitor role. The guest is able to see in the classes that are rated the highest among the students, classes that are rated the lowest among the students, the students with the highest GPA, and college highlight. The guest is also able to submit an application to become either an instructor or student to college that will have to be reviewed by the registrar, if their application is approved. As it can be seen in the image above the guest is submitting an application to become a student.



The image above shows the role of registrar. The registrar is able to decide whether to approve or reject the applications of the visitor that submitted to being instructor or student. They are also able to review complaints that were issued by the student to the instructor and if the student complaints are valid, the registrar can give warnings to the instructor, the registrar can also give warning to the student if in their complaint they use taboo words. These taboo words are words that get filtered out to '***' when written to the complaints and only the instructor can decide which word is considered a taboo word.

EduNavigator – CCNY Intelligent Academic Navigation System	Version: <2.0>
Design Report	Date: 04/23/2026
EduNavigator-SRS-v2.0	

6. Memos of group meetings



We meet on discord and group call for our coding sessions and report discussions.

7. Address of GitHub Repo

https://github.com/memomora07/group_g_EduNavigator

